PLAYWRIGHT ENTERPRISE GUIDE

Build Automation That Survives Reality

Why This Guide Exists

Most Playwright tutorials teach **how to make scripts work**.

This guide teaches **how to make systems that don't break**.

Real-world automation fails because of:

- wrong tool choice
- fragile selectors
- infinite scroll hangs
- session expiry
- crashes with no recovery
- duplicated or corrupted data

This guide eliminates those failures **by design**.

What This Guide Is (and Is Not)

This is NOT

- a Playwright syntax walkthrough
- a scraping cheat sheet
- a "copy–paste and pray" tutorial

This IS

- a systems manual
- a production playbook
- a reliability-first automation guide

You are learning **decision-making**, not button-clicking.

Who This Guide Is For

Read this if you want to:

- run scrapers **unattended**

- handle **JS-heavy, authenticated sites**
- deliver **client-grade automation**
- move from scripts → **data pipelines**
- trust your code after you leave the keyboard

If you want quick hacks, stop here.

How This Guide Works

- 5 days
- each day unlocks **one irreversible capability**
- same structure every day
- binary progress: *done* or *broken*

Skip a day and everything after it fails.

Table of Contents

Day 4 — Browser → API Bridge

10× speed, 10× scale

- Extract tokens
- Kill the browser bottleneck

Day 5 — Production Minimum

From script to system

- Checkpoints
- Resume after crash
- Trustable pipelines

What You Walk Away With

- Automation that **finishes**
- Data that **can be trusted**
- Systems that **recover**
- Skills you can **sell or deploy**

This guide is not about Playwright.

It's about building automation that **survives reality**.

Start with Day 0.
Strategy before code.

DAY 0 — DYNAMIC X-RAY (NO CODE)

Purpose: Decide *how* a website gives data **before** writing code. This choice determines speed, cost, and failure rate.

---

1. Objective (Plain)

Identify the **real data source** of a site in ≤10 minutes.

---

2. Business Meaning (Plain)

- API found → fast, cheap, scalable
- API missed → slow browser automation, higher bans, lower margins Clients pay you to **avoid unnecessary browser work**.

---

3. Mental Model

Website = UI layer + Data layer

Your job is to ignore the UI and locate the **data layer**.

---

4. Execution Checklist (Exact Steps)

1. Open the website normally
2. Open **DevTools → Network**
3. Enable filters: **XHR / Fetch**
4. Perform a real action:
    - scroll
    - search
    - click pagination
5. Watch which requests fire

---

5. Classification (This Is the Core)

Classify the site using this table:

A. Data Source

- **HTML** → data is inside page source
- **XHR JSON API** → `/api/...` returning JSON
- **GraphQL** → `/graphql` with `query`, `edges`, `nodes`

B. Pagination Type

- **Page-based** → `?page=2`
- **Cursor-based** → `cursor=abc123`
- **Infinite scroll** → new requests on scroll

C. Authentication

- **None** → public
- **Cookie-based** → logged-in session
- **Token-based** → headers like `Authorization`, `Bearer`
- **Mixed** → cookie + token

6. Decision Rule (Very Important)

```
If JSON API exists → DO NOT SCRAPE UI
If GraphQL exists → USE API via captured queries
If no API + heavy JS → Playwright
If auth blocks API → Playwright for login, API for data
```

7. Output Artifact (Mandatory)

Write **exactly 4 lines** in notes:

```
Site:
Data Source:
Auth Type:
Best Tool:
```

Example:

```
Site: producthunt.com
Data Source: GraphQL
Auth Type: None
Best Tool: requests / aiohttp
```

8. Failure Modes

- Assuming static HTML when XHR exists
- Missing GraphQL because you didn't scroll
- Choosing Playwright when API was available

9. Acceptance Criteria (Binary)

You can explain **how data flows** from server → screen in **one sentence**.

If not, Day 0 is not done.

DAY 1 — DRIVE THE BROWSER (HUMAN-LEVEL CONTROL)

This day teaches **how to control a browser reliably**, not how to "click buttons with code".

---

1. Objective (Technical + Business)

Technical: Automate a complete user flow (open → interact → reach data) using Playwright **without fragile selectors or timing hacks**.

Business: Replace repetitive human actions with a bot that behaves predictably and survives UI changes.

---

2. Business Value Translation

What clients actually get from Day 1 skills:

- Manual work (copy–paste, clicking filters, navigating pages) becomes **fully automated**
- Automation runs **24/7**, not "when someone is free"
- Reduced human error (missed rows, wrong clicks)
- Lower long-term maintenance cost (script doesn't break every week)

If your automation breaks on small UI changes, it is **not sellable**.

---

3. Mental Model

Browser = Low-IQ Human Intern

A browser does not:

- "understand intent"
- "guess timing"
- "adapt"

You must:

- Tell it *exactly what element matters*
- Tell it *exactly when to act*
- Remove *all ambiguity*

If instructions are vague → automation is fragile.

---

4. Execution Checklist (Physical Actions)

Follow these **in order**, no skipping.

1. **Install Playwright**

```
python -m pip install playwright
playwright install
```

   - Installs Playwright library
   - Downloads real browsers (Chromium, Firefox, WebKit)

2. **Run Codegen**

```
playwright codegen https://example.com
```

   What this does:

   - Opens a real browser
   - Records your clicks, typing, scrolling
   - Generates working Playwright code

3. **Record a Full Flow** Example flow:

   - Open site
   - Search / click category
   - Navigate to data-visible page

4. **Manual Cleanup (Mandatory)** Delete:

   - All `time.sleep(...)`
   - All deep `XPath` selectors

---

5. Pipeline (System Flow)

```
Human Demonstration
→ Codegen Script
→ Selector Replacement
→ Deterministic Replay
```

Explanation:

- **Human Demonstration:** you show what a real user does
- **Codegen Script:** Playwright records raw actions

- **Selector Replacement:** you replace fragile selectors
- **Deterministic Replay:** script runs the same way every time

---

6. Core Syntax Ownership (Explained Clearly)

A. Selectors (MOST IMPORTANT CONCEPT)

Selectors tell Playwright **how to identify elements**.

1. ❌ XPath (What NOT to use)

Example:

```
page.locator('/html/body/div[3]/span[2]')
```

Why XPath fails:

- Depends on layout structure
- Breaks when a div is added/removed
- UI redesign = script dead

Use XPath only as last resort.

---

2. ☑ Role-Based Selectors (Best)

Example:

```
page.get_by_role("button", name="Search")
```

What this means:

- `role="button"` → semantic meaning, not position
- `name="Search"` → visible label

Why it's stable:

- Based on accessibility tree
- Survives layout changes

---

3. ☑ Text-Based Selectors

Example:

```
page.get_by_text("Login")
```

Use when:

- Text is visible
- Text is unique

Risk:

- Breaks if text changes language/wording

---

4. ☑ Attribute Selectors

Example:

```
page.locator('[data-testid="submit"]')
```

Best when:

- Site uses `data-*` attributes
- Attributes are stable by design

B. Timing Control (Second Most Important Concept)

✗ `time.sleep()`

```
time.sleep(5)
```

Why it's bad:

- Guesswork
- Too slow or too fast
- Causes flaky scripts

☑ Playwright Auto-Waits

Playwright automatically waits for:

- Elements to appear
- Elements to be clickable
- Navigation to finish

Example:

```
page.click("button")
```

Playwright waits until button is ready.

---

7. Failure Modes (Real-World Breakpoints)

- UI update changes div structure → XPath breaks
- Page loads slower one day → `sleep(2)` fails
- Multiple similar buttons → vague selector clicks wrong one

These failures **always happen in production**.

---

8. Acceptance Criteria (“Done When”)

Day 1 is complete **only if all are true**:

- Script contains **zero** `time.sleep`
- Script contains **no deep XPath**
- Script runs:
    1. Once
    2. Refresh page
    3. Runs again
- Output behavior is identical

If any condition fails → Day 1 is not done.

---

9. Output Artifact

A **stable browser automation script** that:

- Mimics a real human flow

- Uses resilient selectors
- Runs repeatedly without modification

This script becomes the **foundation** for:

- scrolling logic (Day 2)
- login reuse (Day 3)
- API extraction (Day 4)
- production systems (Day 5)

Day 1 is about **control**. If you don't fully control the browser, nothing after this scales.

DAY 2 — STABILITY ENGINEERING (SCROLLS & WAITS)

This day is about **making automation safe**. Not faster. Not smarter. **Safe.**

Most scraping failures in the real world happen here.

---

1. Objective (Technical + Business)

Technical: Ensure the script:

- never hangs forever
- never exits too early
- always knows *when to stop*

Business: Clients depend on **predictable completion**. A script that sometimes finishes and sometimes hangs is **unusable in production**.

---

2. Business Value Translation

What instability costs in real life:

- Hung scraper → server resources wasted
- Partial data → corrupted dashboards
- Silent failure → client trusts bad data
- Manual babysitting → defeats automation

Day 2 turns a *demo script* into a **production-safe worker**.

---

3. Mental Model

Scraper = State Machine

The scraper must always be in **one of two states**:

- State changed → continue
- State unchanged → stop

You are not "scrolling until looks done". You are **detecting whether the system state is evolving**.

---

4. Execution Checklist (Physical Actions)

You must do **all** of the following:

1. Remove **all** remaining `time.sleep`
2. Replace sleeps with **explicit waits**
3. Implement a **stop condition**
4. Add **failure evidence**:
    - screenshot
    - HTML dump
    - optional trace

---

5. Pipeline (System Flow)

```
Trigger Load (scroll / click)
→ Wait for condition
→ Measure state
→ Compare with previous state
→ Decide: continue OR stop
→ On error: save evidence
```

This loop must be **finite by design**.

---

6. Core Syntax Ownership (Explained Clearly)

A. Explicit Waits (What They Are)

An **explicit wait** means:

> "Pause execution until a specific condition becomes true."

Not time-based. Condition-based.

---

1. `wait_for_selector`

```
page.wait_for_selector(".item")
```

Meaning:

- Pause until an element matching `.item` exists in DOM

Use when:

- Content appears after loading
- You know *what element signals readiness*

Risk:

- Selector appears but data inside isn't complete

---

2. `wait_for_load_state("networkidle")`

```
page.wait_for_load_state("networkidle")
```

Meaning:

- Wait until no network requests for ~500 ms

Use when:

- Page loads data via XHR
- Network activity clearly maps to loading

Do **not** use when:

- Page polls continuously
- Analytics fires background requests

B. Infinite Scroll (Core Problem)

Infinite scroll pages **never tell you they are done**.

So you must detect completion yourself.

C. Stop Condition (Most Important Concept)

A **stop condition** is a rule that proves:

> "Nothing new is loading anymore."

Example strategy: **item count comparison**

```
prev_count = 0

while True:
    page.mouse.wheel(0, 15000)
    page.wait_for_timeout(1000)  # controlled, short wait

    items = page.query_selector_all(".item")
    current_count = len(items)

    if current_count == prev_count:
        break  # state unchanged → stop

    prev_count = current_count
```

What this does:

- Scrolls
- Counts items
- Stops when count no longer increases

D. DOM Mutation vs Data Mutation

Important distinction:

- **DOM mutation**: new elements added
- **Data mutation**: same DOM nodes, new data injected

If DOM doesn't change:

- Track **unique item IDs**, not element count

Example logic:

```
seen_ids = set()

for item in items:
    seen_ids.add(item.get_attribute("data-id"))
```

Stop when `seen_ids` stops growing.

E. Failure Evidence (Non-Negotiable)

On any exception, save:

```
page.screenshot(path="crash.png")
html = page.content()
open("crash.html", "w").write(html)
```

Why:

- Debug without rerunning
- Prove failure cause
- Audit client issues

---

7. Failure Modes (Real-World Breakpoints)

- Lazy loading updates text but not DOM → count logic fails
- Network jitter delays load → premature stop
- Continuous polling → `networkidle` never triggers
- Missing evidence → impossible debugging

These are **normal**, not edge cases.

---

8. Acceptance Criteria (“Done When”)

Day 2 is complete **only if**:

- Script **always exits** (never infinite loop)
- No `time.sleep` except **short, controlled waits**
- Stop condition is **data-based**, not visual guess
- On failure:

- screenshot exists
- HTML dump exists

If script can hang → Day 2 failed.

---

9. Output Artifact

A **self-terminating, observable scraper** that:

- Knows when to stop
- Fails loudly
- Leaves forensic evidence
- Can run unattended

This is the **minimum requirement** for:

- scheduled jobs
- long runs
- real clients

Day 2 is where most beginners fail. If this day is solid, **everything after it becomes possible**.

DAY 3 — AUTH & SESSION ENGINEERING

This day is about **making your automation behave like a real returning user**, not a suspicious bot.

Most real-world scraping projects fail here.

---

1. Objective (Technical + Business)

Technical: Log in **once**, reuse the authenticated session across runs, detect expiry, and recover automatically.

Business: Avoid bans, CAPTCHAs, wasted compute, and fragile pipelines that break every few hours.

A system that cannot handle authentication **cannot run unattended**.

---

2. Business Value Translation

What poor auth handling costs in real life:

- Logging in every run → triggers security systems
- Frequent CAPTCHAs → manual intervention required
- Session loss → broken daily jobs
- Higher proxy and infra cost

What Day 3 enables:

- Long-running pipelines
- Human-like behavior
- Lower detection risk
- Predictable delivery to clients

Clients don't pay for scripts that need babysitting.

---

3. Mental Model

Session = Asset Login = Liability

- Login is noisy, slow, and suspicious
- Sessions are quiet, stable, and human-like

Your job is to **minimize logins** and **maximize session reuse**.

---

4. Execution Checklist (Physical Actions)

You must complete **all** steps in this order:

1. Run browser in **headed mode** (`headless=False`)
2. Perform login (manual or scripted)
3. Save browser storage state to disk
4. Re-launch browser using saved state
5. Detect when session expires
6. Re-login and overwrite stored state
7. Resume execution from where you stopped

Skipping any step breaks the system.

---

5. Pipeline (System Flow)

```
Login (once)
→ Save Session State
→ Reuse Session
→ Detect Expiry
```

```
→ Refresh Session
→ Resume Work
```

This pipeline must work **without restarting the whole job**.

---

6. Core Syntax Ownership (Explained Clearly)

A. What "Authentication" Actually Means

When you log in, the server gives the browser **proof** that you are authenticated. This proof lives in:

- **Cookies** (most common)
- **localStorage / sessionStorage**
- Sometimes both

As long as this proof exists and is valid, you are "logged in".

B. `storage_state` (Core Concept)

Playwright can **export and import** this proof.

Saving auth state

```
ctx.storage_state(path="auth.json")
```

What this does:

- Saves cookies
- Saves localStorage
- Saves sessionStorage
- Writes everything to `auth.json`

This file represents **a logged-in browser identity**.

Reusing auth state

```
browser.new_context(storage_state="auth.json")
```

Meaning:

- Browser starts **already logged in**
- Login page is skipped
- Session behaves like a returning user

This is the single most important Playwright feature for production work.

C. Headed vs Headless (Why Headed Is Required Initially)

- **Headed (`headless=False`)**
    - Visible browser
    - Required for:
        - manual login
        - CAPTCHA solving
        - 2FA approval
- **Headless**
    - Faster
    - Used after session is saved

You do **not** brute-force auth in headless mode.

D. Session Expiry (What It Is)

Sessions are **temporary**.

They expire because of:

- Time limits (TTL)
- Server restarts
- Manual logout
- Security policies

When expired, the server silently treats you as logged out.

E. Detecting Session Expiry (Critical Skill)

Common signals:

1. Redirect to `/login`

2. Login form selector appears
3. API responses return `401` or `403`

Example logic (conceptual):

```
if page.url.endswith("/login"):
    session_expired = True
```

or

```
if page.locator("input[type=password]").is_visible():
    session_expired = True
```

Detection must happen **before data extraction continues**.

F. Session Refresh Logic

When expiry is detected:

1. Pause automation
2. Re-login (manual or scripted)
3. Overwrite `auth.json`
4. Reload context
5. Resume pipeline

This keeps long jobs alive.

---

7. Failure Modes (Real-World Breakpoints)

- 2FA appears → script loops forever
- Session expires mid-run → partial data
- Auth state not updated → repeated failures
- Multiple accounts share same auth file → undefined behavior

These failures are **normal**, not rare.

---

8. Acceptance Criteria (“Done When”)

Day 3 is complete **only if**:

- Login happens **once**
- `auth.json` is created
- Script runs using stored session
- You manually delete cookies / auth file
- Script:
    - detects logout
    - re-authenticates
    - resumes execution

If restart is required → Day 3 failed.

---

9. Output Artifact

A **reusable authentication state file** and logic that:

- Mimics real user behavior
- Survives session expiry
- Enables unattended execution
- Unlocks scaling (Day 4)

Day 3 is the line between:

- hobby scripts
- professional automation systems

If authentication is weak, **everything after this collapses**.

DAY 4 — BROWSER → API BRIDGE (SCALE)

This day is about **escaping the browser bottleneck**. You stop *using* Playwright for data and start *using it as a key*.

---

1. Objective (Technical + Business)

Technical: Use Playwright only to:

- pass login / security
- discover real API calls
- extract auth headers & tokens

Then fetch data at scale using **direct HTTP requests** (`aiohttp`).

Business: Browsers are slow and expensive. APIs are fast and cheap.

Day 4 is where **enterprise-scale throughput** becomes possible.

---

2. Business Value Translation

What staying browser-only costs:

- Each page load = seconds
- High CPU/RAM usage
- Limited concurrency
- Higher proxy & infra bills

What Day 4 unlocks:

- 10×–50× faster data extraction
- Massive concurrency
- Lower detection surface
- Ability to serve large clients and long jobs

Clients don't pay for "headless Chrome farms". They pay for **fast, quiet data pipelines**.

---

3. Mental Model

Browser = Key Generator API = Data Firehose

The browser's job ends the moment:

- tokens are captured
- endpoints are known

After that, the browser is a liability.

---

4. Execution Checklist (Physical Actions)

You must complete **all** steps:

1. Run authenticated Playwright session (from Day 3)
2. Observe **Network → XHR / Fetch**
3. Identify JSON responses that contain real data
4. Log:

- request URL
- request headers
- cookies
- auth tokens

5. Replicate the same request using `aiohttp`
6. Fetch multiple pages concurrently
7. Compare speed vs browser-based extraction

---

5. Pipeline (System Flow)

```
Browser Login
→ Observe Network Traffic
→ Identify Real API Endpoint
→ Extract Headers / Tokens
→ Rebuild Request
→ Async API Fetch (Scale)
```

Once this pipeline works, Playwright becomes **optional**.

---

6. Core Syntax Ownership (Explained Clearly)

A. What “Network Interception” Means

Network interception = **watching every request and response the browser makes**.

Playwright allows you to listen to these events.

B. Listening to Responses

Conceptual example:

```
def on_response(response):
    if "application/json" in response.headers.get("content-type",
""):
        print(response.url)

page.on("response", on_response)
```

What this does:

- Runs for **every response**
- Filters only JSON responses
- Shows real data endpoints

This is how you discover:

- hidden APIs
- GraphQL endpoints
- pagination parameters

C. Identifying the "Real" API

Ignore:

- analytics
- logging
- ads
- feature flags

Look for:

- responses returning lists of items
- fields you care about (id, name, price, etc.)
- pagination params (`page`, `cursor`, `offset`)

This API is the **data source**.

D. Headers (Critical Concept)

APIs often require **specific headers**.

Common ones:

- `Authorization`
- `Cookie`
- `X-CSRF-Token`
- `User-Agent`
- `Referer`

If headers don't match → request fails.

You must **mirror headers exactly**.

E. Tokens (What They Are)

A **token** is proof that:

- you logged in
- you are authorized
- the request is allowed

Tokens can live in:

- headers
- cookies
- localStorage

Tokens often:

- expire
- are user-specific
- are tied to session identity

F. Replaying Requests with `aiohttp`

`aiohttp` is an **async HTTP client**.

Why async matters:

- multiple requests run at the same time
- no browser overhead
- orders of magnitude faster

Conceptual idea:

```
async with aiohttp.ClientSession(headers=headers, cookies=cookies)
as session:
    async with session.get(url) as resp:
        data = await resp.json()
```

This is now **pure data fetching**, not automation.

G. Why This Scales

Browser:

- 1 page = 1 tab

- Heavy memory usage
- Sequential behavior

API + async:

- 100+ concurrent requests
- Minimal resources
- Horizontal scaling

This is the difference between:

- hobby scraping
- production pipelines

---

7. Failure Modes (Real-World Breakpoints)

- Missing a required header → 403 errors
- Token expires mid-run → silent failures
- Token bound to browser fingerprint → API replay blocked
- Pagination params misunderstood → duplicate or missing data

When these happen:

- fall back to browser fetch
- or refresh tokens via Playwright

---

8. Acceptance Criteria (“Done When”)

Day 4 is complete **only if**:

- Browser is used **only** for login/token capture
- Same data is fetched via API without UI
- API method is **≥10× faster** than browser loop
- Pagination works fully via API
- Data matches browser-extracted data

If browser is still scrolling → Day 4 failed.

---

9. Output Artifact

A **hybrid extraction system** where:

- Playwright handles access & security
- `aiohttp` handles data at scale

- Browser usage is minimal and controlled
- Pipeline is fast, quiet, and scalable

Day 4 is the turning point.

Before this: automation engineer. After this: **data pipeline engineer**.

Everything in Day 5 assumes **this capability exists**.

DAY 5 — PRODUCTION MINIMUM (SYSTEMIZATION)

This day turns **working code** into a **system you can trust**. Everything before this was about *capability*. Day 5 is about *reliability under failure*.

---

1. Objective (Technical + Business)

Technical: Convert individual scripts into a **resumable, self-healing pipeline** that can survive crashes, restarts, and partial failures.

Business: Clients and production systems do not care if your script works *once*. They care that it works **every day without supervision**.

---

2. Business Value Translation

What non-systemized scripts cost in reality:

- Crash after 6 hours → all progress lost
- Partial data → broken analytics
- Manual restarts → human dependency
- No logs → no accountability

What Day 5 delivers:

- “Set and forget” automation
- Crash tolerance
- Predictable outputs
- Professional-grade delivery

This is the difference between:

- a freelancer demo
- a billable system
- a real product

---

3. Mental Model

Failure is expected. Recovery is mandatory.

Production systems assume:

- servers crash
- networks fail
- sessions expire
- processes get killed

Your job is not to *prevent* failure. Your job is to **resume after failure without damage**.

---

4. Execution Checklist (Physical Actions)

You must implement **all** of the following:

1. Enforce a clear folder structure
2. Add structured logging with timestamps
3. Implement a checkpoint mechanism
4. Ensure idempotent data writes
5. Validate extracted data
6. Test crash + resume behavior
7. Test session expiry + resume behavior

Skipping any step makes the pipeline unsafe.

---

5. Pipeline (System Flow)

```
Input
→ Load Auth
→ Fetch Data
→ Validate
→ Save Output
→ Save Checkpoint
→ Resume on Restart
```

The pipeline must be restartable from **any step**.

---

6. Core Syntax Ownership (Explained Clearly)

A. Folder Structure (Why It Matters)

A predictable structure enforces discipline:

```
project/
├── config/
├── logs/
├── auth/
├── data/
│   ├── raw/
│   ├── processed/
│   └── errors/
├── checkpoints/
└── main.py
```

Meaning:

- `config/` → constants, URLs, limits
- `logs/` → execution history
- `auth/` → session files (`auth.json`)
- `data/raw/` → untouched fetched data
- `data/processed/` → clean output
- `data/errors/` → invalid rows
- `checkpoints/` → resume state

This prevents silent chaos.

B. Logging (What "Structured Logging" Means)

Logging is not `print()`.

A log entry answers:

- when did this happen
- what step failed
- what was being processed

Example concept:

```
2026-02-15 14:32:10 | INFO | fetching page 12
2026-02-15 14:32:14 | ERROR | session expired
```

Logs are:

- evidence
- debugging tools
- client-facing accountability

C. Checkpoints (Core Concept)

A **checkpoint** is a saved snapshot of progress.

Typical checkpoint data:

- last page number
- last item ID
- last cursor value

Example concept:

```
{
  "last_page": 12,
  "last_item_id": "abc123"
}
```

Checkpoints are written **after successful work**, not before.

D. Resume Logic (Why It's Hard)

On restart:

1. Read checkpoint
2. Skip already-processed items
3. Continue from saved state
4. Avoid duplicates

This requires **idempotency**.

E. Idempotency (Critical Term)

Idempotent means:

> Running the same step twice does **not** create duplicates or corruption.

Example:

- Writing output keyed by unique ID

- Skipping items already saved

Without idempotency, resume = data corruption.

F. Data Validation (Why It's Mandatory)

Scraped data is often:

- incomplete
- malformed
- partially missing

Validation means:

- checking required fields
- checking data types
- separating bad rows

Bad data goes to:

```
data/errors/errors.json
```

Good data continues.

G. Session Handling in Production

Session expiry must not:

- kill the pipeline
- corrupt checkpoints

Correct behavior:

1. Detect expiry
2. Refresh auth
3. Resume from checkpoint

Auth refresh is **orthogonal** to data progress.

---

7. Failure Modes (Real-World Breakpoints)

- Process killed mid-write → corrupted file
- Checkpoint saved too early → skipped data

- No idempotency → duplicate rows
- Logs missing → no post-mortem possible
- Validation skipped → silent bad data

These are **production failures**, not edge cases.

---

8. Acceptance Criteria (“Done When”)

Day 5 is complete **only if**:

- Pipeline runs end-to-end
- You kill the process manually
- Restart the script
- Pipeline:
  - loads checkpoint
  - resumes correctly
  - produces no duplicates
- Session expiry is handled automatically
- Logs clearly show failure and recovery

If restart requires manual intervention → Day 5 failed.

---

9. Output Artifact

A **trustable, unattended data pipeline** that:

- Survives crashes
- Resumes without corruption
- Handles auth expiry
- Produces validated outputs
- Can be scheduled or deployed

Day 5 is the finish line.

Before this: scripts that *work*. After this: systems you can **sell, deploy, and trust**.

This completes the guide.